



Python程序语法元素分析

崔海

大连海事大学

cuihai@dlmu.edu.cn



程序的格式框架

- 程序设计的6个步骤是利用计算机解决问题的方法步骤，程序设计语言则是解决问题的实现载体
- Python采用严格的“**缩进**”来表明程序的格式框架
 - 缩进指每一行代码开始前的空白区域，用来表示代码之间的**包含**和**层次**关系
 - ✓ 可以用Tab键实现
 - ✓ 也可以用多个空格(一般是4个空格)实现
 - 严格的缩进有利于维护代码结构的可读性

程序的格式框架

- Python采用严格的“**缩进**”来表明程序的格式框架

- 单层缩进

```
i = 0
while i < 3:
    print(i)
    i = i + 1
```

```
print('结束')
```



```
0
1
2
结束
```

```
i = 0
while i < 3:
    print(i)
    i = i + 1
```

```
print('结束')
```



```
0
结束
1
结束
2
结束
```

程序的格式框架

- Python采用严格的“**缩进**”来表明程序的格式框架

- 单层缩进

```
i = 0
while i < 3:
    print(i)
    i = i + 1
```

```
print('结束')
```

```
i = 0
while i < 3:
    print(i)
    i = i + 1
```

```
print('结束')
```

两段代码一个字都没改，只挪了一下位置，结果完全不同。

- 左侧：`print('结束')` 顶格写，不属于循环，等循环全跑完才执行一次
- 右侧：`print('结束')` 缩进了，属于循环内部，每转一圈都执行一次

缩进决定了一行代码“归谁管”。

程序的格式框架

- Python采用严格的“**缩进**”来表明程序的格式框架

- 多层缩进**

```
i = 0
```

```
while i < 3:
```

```
    if i == 1:
```

```
        print('    找到 1 了')
```

```
    print(i)
```

```
    i = i + 1
```



0

找到 1 了

1

2

缩进	代码	归谁管
0 格	<code>while i < 3:</code>	最外层
4 格	<code>if i == 1:</code>	归 <code>while</code> 管，每轮判断一次
8 格	<code>print('找到1了')</code>	归 <code>if</code> 管，条件成立才执行
4 格	<code>print(i)</code>	归 <code>while</code> 管，每轮都执行
4 格	<code>i = i + 1</code>	归 <code>while</code> 管，每轮都执行

缩进多少格，
就决定了它属于哪一层

■ 注释：程序员在代码中加入的说明信息

- 用来对语句、函数、数据结构或方法等进行说明，提升代码的可读性
- 不被计算机执行

■ 两种注释方法：

- 单行注释：以 # 开头
- 多行注释：以 3 个 **单引号** (或**双引号**) 开头和结尾



程序注释

这是单行注释

这是多行注释的第一种写法

每一行前面都加一个

这是最标准的做法

"""

这是多行注释的第二种写法

用三个双引号把内容包起来

中间可以随便换行

"""

'''

三个单引号也可以

效果完全一样

'''

`print('Hello')`

井号也可以写在代码后面

■ 注释的三个用途

- 标明作者和版权信息：一般用在文件开头
- 解释代码原理或用途：解释关键代码作用，增加可读性
- 辅助程序调试：通过注释“去掉”与当前调试无关的代码



命名与保留字

Python程序采用**变量**来保存和表示特定的数据值，为了更好地使用变量，需要给它们关联一个标识符(名字)，关联标识符的过程称为命名。变量的类型和值在赋值的那一刻被初始化。

- Python语言允许采用**大写字母、小写字母、数字、下划线_和汉字**等字符及其组合给变量命名
- 但名字的首字符不能是数字，中间不能出现空格，长度没有限制：

`python_`、`_python`、`python_123`

⚠ 标识符对大小写敏感，`python_`和`Python_`是两个不同的名字；从编程习惯和兼容性角度考虑，一般**不建议**采用中文等非英语语言字符对变量命名

- 为什么变量命名不能以数字开头？

123 = 456

无法区分是变量名还是数字常量

例如：如果变量名123合法，则程序无法区分这个是变量123，还是数字常量123

- Python 3 允许汉字作为变量名、函数名、参数名

```
半径 = 5  
面积 = 3.14 * 半径 ** 2  
print(面积)
```

```
>>> def 求和(甲, 乙):  
...     return 甲 + 乙  
>>> 求和(3, 5)  
8
```

- **保留字**：也称为关键字，指被编程语言内部定义并保留使用的标识符
- 程序员编写程序**不能**定义与保留字相同的标识符
- 每种程序设计语言都有一套保留字，保留字一般用来构成程序整体框架、表达关键值和具有结构性的复杂语义等
- 掌握一门编程语言首先要熟记其所对应的保留字



命名与保留字

■ Python 3 保留字列表 (35个)

```
>>> import keyword
>>> keyword.kwlist
['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await', 'break',
'class', 'continue', 'def', 'del', 'elif', 'else', 'except', 'finally',
'for', 'from', 'global', 'if', 'import', 'in', 'is', 'lambda', 'nonlocal',
'not', 'or', 'pass', 'raise', 'return', 'try', 'while', 'with', 'yield']
```



命名与保留字

■ Python 3 保留字列表 (35个)

值与常量 (3 个)

关键字	解释
True	真
False	假
None	空值, 表示"什么都没有"

条件判断 (3 个)

关键字	解释
if	如果.....就.....
elif	否则如果 (else if 的缩写)
else	否则

循环控制 (4 个)

关键字	解释
for	依次遍历每一项
while	当条件成立时一直重复
break	立刻跳出整个循环
continue	跳过本次, 进入下一轮

逻辑运算 (3 个)

关键字	解释
and	并且, 两边都成立才为真
or	或者, 有一边成立就为真
not	取反, 真变假、假变真



命名与保留字

■ Python 3 保留字列表 (35个)

成员与身份判断 (2个)

关键字	解释
<code>in</code>	在不在里面, 如 <code>'a' in 'abc'</code>
<code>is</code>	是不是同一个对象

函数 (4个)

关键字	解释
<code>def</code>	定义一个函数
<code>return</code>	返回结果并结束函数
<code>lambda</code>	定义一个匿名的小函数
<code>yield</code>	生成器专用, 逐个产出值

类 (1个)

关键字	解释
<code>class</code>	定义一个类

异常处理 (5个)

关键字	解释
<code>try</code>	尝试执行, 可能会出错
<code>except</code>	出错时执行这里
<code>finally</code>	不管出没出错都要执行
<code>raise</code>	主动抛出一个错误
<code>assert</code>	断言, 条件不成立就报错



命名与保留字

■ Python 3 保留字列表 (35个)

模块导入 (3 个)

关键字	解释
<code>import</code>	导入模块
<code>from</code>	从某个模块中导入
<code>as</code>	起个别名, 如 <code>import numpy as np</code>

作用域 (2 个)

关键字	解释
<code>global</code>	声明使用全局变量
<code>nonlocal</code>	声明使用外层函数的变量

其他 (3 个)

关键字	解释
<code>pass</code>	占位, 什么都不做
<code>del</code>	删除变量或元素
<code>with</code>	自动管理资源, 如打开文件后自动关闭

异步编程 (2 个)

关键字	解释
<code>async</code>	定义异步函数
<code>await</code>	等待异步操作完成



多选题

■ 下面变量命名合法的是？

A. 1count

B. count123

C. 我的变量123

D. _123

E. my count

F. class

多选题

■ 下面变量命名合法的是？

A. 1count

B. count123

C. 我的变量123

D. _123

D. my count

E. class

- 可以使用 `isidentifier()` 用来判断 “字符串能不能当变量名”，返回 `True` 或 `False`

```
>>> '1count'.isidentifier()
False
>>> 'count123'.isidentifier()
True
>>> '我的变量123'.isidentifier()
True
>>> '_123'.isidentifier()
True
>>> 'my count'.isidentifier()
False
```

⚠ `isidentifier()` 只检查 “字符组合的形式对不对”，不检查 “这个词是不是被 Python 占用了”

```
>>> 'class'.isidentifier()
True
```

注意！形式上合法，但它是关键字，实际不能用

字符串

- 在Python语言中，字符串是用两个双引号 “ ” 或者单引号 ‘ ’ 括起来的一个或多个字符
- 字符串是字符的序列，可以按照单个字符或字符片段进行索引，包括两种序号体系



正向递增最左侧字符序号为 0，向右依次递增；
反向递减最右侧字符序号为 -1，向左依次递减；
两种字符方法可以同时使用

■ 索引取单个字符

```
>>> s = 'Hello World'
```

```
>>> len(s)                                # 字符串的长度
```

```
11
```

```
>>> s[0]                                # 正向：第一个字符
```

```
'H'
```

```
>>> s[6]
```

```
'W'
```

```
>>> s[-1]                                # 反向：最后一个字符
```

```
'd'
```

⚠ $s[-1] = s[\text{len}(s)-1]$, 但 $s[-1]$ 更好, 因为无需事先知道字符串的长度

■ 切片取一段

```
>>> s[0:5]
```

```
'Hello'
```

```
>>> s[6:11]
```

```
'World'
```

```
>>> s[:5]
```

```
'Hello'
```

```
>>> s[6:]
```

```
'World'
```

```
>>> s[:]
```

```
'Hello World'
```

从 0 开始, 到 5 之前结束

⚠ s[0:5] 取的是索引 0、1、2、3、4 号, 不包括 5 号

开头可以省略

结尾也可以省略

都省略 = 整个字符串

赋值语句

- Python语言中，`=` 表示“赋值”，即将等号右侧的值计算后将结果值赋给左侧变量
- 包含等号（`=`）的语句称为“赋值语句”

`<变量1> = <表达式1>` →

产生或计算新数据值的代码称为表达式

```
>>> a = 3 + 5
```

```
>>> a
```

```
8
```

```
>>> radius = 2
```

```
>>> area = 3.14 * radius * radius
```

```
>>> area
```

```
12.56
```

赋值语句

■ 同步赋值语句：同时给多个变量赋值

$\langle \text{变量1} \rangle, \dots, \langle \text{变量N} \rangle = \langle \text{表达式1} \rangle, \dots, \langle \text{表达式N} \rangle$

同步赋值并非等同于简单地将多个单一赋值语句进行组合

Python在处理同步赋值时首先运算右侧的N个表达式，同时将表达式的结果赋值给左侧的N个变量

例：将变量x和y交换

■ 采用单个赋值，需要3行语句；采用同步赋值语句，仅需要一行代码：

<pre>>>>t=x</pre>		<pre>>>>x, y=y, x</pre>
<pre>>>>x=y</pre>		
<pre>>>>y=t</pre>		

赋值语句

■ 同步赋值语句：同时给多个变量赋值

例：斐波那契数列

在数学上，斐波那契数是以递归的方法来定义：

- $F_0 = 0$
- $F_1 = 1$
- $F_n = F_{n-1} + F_{n-2} \quad (n \geq 2)$

```
>>> a, b = 0, 1
>>> for i in range(8):
...     print(a, end=' ')
...     a, b = b, a + b
...
0 1 1 2 3 5 8 13
```

赋值语句

- 同步赋值语句：同时给多个变量赋值

= 左右两边数量必须相等，否则报错：

```
>>> m, n = 1, 2, 3
```

```
ValueError: too many values to unpack (expected 2)
```



input()函数

- input()函数从控制台获得用户输入，以字符串类型返回结果
- 获得用户输入之前，input()函数可以包含一些提示性文字

<变量> = input(<提示性文字>)

```
>>> s = input('请输入: ')
```

```
请输入: python
```

```
>>> s
```

```
'python'
```

```
>>> t = input('请输入: ')
```

```
请输入: 123.45
```

```
>>> t
```

```
'123.45'
```



input()函数

```
>>> a = input('第一个数: ')
```

第一个数: 3

```
>>> b = input('第二个数: ')
```

第二个数: 5

```
>>> a + b
```

问题：输出是多少？

答案： '35' 而非 8 ！

input()函数

- input() 函数 无论输入什么，返回的都是字符串

python

```
>>> x = input('请输入一个数字: ')
```

```
请输入一个数字: 5
```

```
>>> x
```

```
'5'
```

注意引号

```
>>> type(x)
```

```
<class 'str'>
```

是字符串，不是整数

为什么上一页答案是 '35' 而非 8？

a 是字符串 '3'，b 是字符串 '5'，字符串的 + 是拼接，不是加法！



input()函数

■ 解决方案

先 input() 拿字符串，再 int() 转成整数，最后赋给 a

```
a = int(input('第一个数: '))  
b = int(input('第二个数: '))  
print(a + b)
```

⚠ 转换失败会报错

```
>>> n = int(input('请输入数字: '))
```

请输入数字: abc

```
ValueError: invalid literal for int() with base 10: 'abc'
```

eval() 函数

- `eval(<字符串>)` 函数是能够以 Python 表达式的方式解析并执行字符串，将返回结果输出

```
>>> tempX = 1
>>> eval('tempX + 1')
2
>>> eval('1.2 + 1.3')
2.5
```

一句话概括 `eval` 的作用是：去掉引号，把字符串变成真正的代码

⚠ `eval` 只接受表达式（能算出一个值），不接受语句

```
>>> eval('x = 5')                                # × 赋值语句不行
SyntaxError: invalid syntax
```

eval() 函数

- 使用eval()函数处理字符串需要注意合理使用

```
>>> eval('hello')
```

```
NameError: name 'hello' is not defined
```

eval 去掉引号后，hello 就变成了一个**变量名**。因为之前没定义过这个变量，所以报错



```
>>> hello = 123
```

先定义这个变量

```
>>> eval('hello')
```

```
123
```

现在能取到它的值了



```
>>> eval("'hello'")
```

注意：外层双引号，内层单引号

```
'hello'
```

去掉外层引号后，里面还剩一对引号，'hello' 就是一个字符串，所以返回字符串本身

eval() 函数

■ 将 input() 和 eval() 函数结合使用

```
>>> value = eval(input('请输入要计算的数值: '))
```

请输入要计算的数值: 123.45

```
>>> print(value * 2)
```

246.9

⚠ 用户输入的内容会被原样执行，存在安全风险

```
value = eval(input('请输入数字: '))
```

```
print('结果: ', value)
```

请输入数字: __import__("os").getcwd()

用户输入的根本不是数字，却成功执行了系统命令、拿到了当前路径。换成删除文件的命令，程序就真的会删！

print()函数

- print()函数用来输出字符信息，或以字符形式输出变量

`print(*objects, sep=' ', end='\n', file=sys.stdout, flush=False)`

- **objects**--表示输出的对象（多个对象时，需要用逗号分隔）
- **sep**--设置在输出值之间的间隔字符，默认是空格
- **end**--设定以什么结尾，默认是换行符
- **file**--指定输出的目标，默认为控制台(`sys.stdout`)，可以将输出重定向到文件或其他对象
- **flush**--控制输出缓冲，一般保持 `False` 即可

```
>>> print("hello world!")  
hello world!
```

```
>>> print("你好", "世界", sep='-', end='!')  
你好-世界!
```

```
>>> print("你好", "世界", sep='-', end='\n')  
你好-世界
```

```
>>> with open("output.txt", "w") as f:  
...     print("你好", "世界", file=f)
```

print()函数

- 当输出变量值时，需采用格式化输出方式，通过**format()** 方法将待输出变量整理成期望输出的格式

- 基本用法

```
print('我叫{}, 今年{}岁'.format('小明', 18))
```

```
# 我叫小明, 今年18岁
```

```
print('{}+{}={} '.format(3, 5, 3+5))
```

```
# 3+5=8
```

{} 是占位符，.format() 里的值按顺序往其中填入

print()函数

- 当输出变量值时，需采用格式化输出方式，通过**format()** 方法将待输出变量整理成期望输出的格式
- 控制小数位数

```
print('圆周率是 {:.2f}'.format(3.14159))
```

```
# 圆周率是 3.14
```

```
print('{:.0f}'.format(3.14159))
```

```
# 3
```

```
print('{:.4f}'.format(3.14159))
```

```
# 3.1416
```

冒号后面是格式说明：**.2** 表示保留 2 位，**f** 表示按小数显示。会进行四舍五入

print()函数

- 当输出变量值时，需采用格式化输出方式，通过**format()** 方法将待输出变量整理成期望输出的格式

- 实用小技巧

```
print('{:,}'.format(1234567))
```

千位分隔符

```
# 1,234,567
```

```
print('{:.1%}'.format(0.856))
```

转成百分比

```
# 85.6%
```

分支语句

■ 分支语句用于根据判断条件选择程序执行路径

<code>if <条件1>:</code> <语句块1>	如果 条件1 成立 就执行 语句块1
<code>elif <条件2>:</code> <语句块2>	否则, 如果 条件2 成立 就执行 语句块2
...	可以有任意多个 elif
<code>else:</code> <语句块N>	以上都不成立 就执行 语句块N

: 和缩进 每个条件行末尾都要写冒号，下一行必须缩进——**缩进的部分才归这个分支管**

执行顺序 从上往下依次判断，**命中一个就执行，后面的全部跳过**。所以最多只有一个语句块会被执行

可选部分 elif 可以有多个，也可以一个都没有；else 最多一个，也可以省略。只有 if 是必需的



分支语句

■ 分支语句用于根据判断条件选择程序执行路径

score = 85

```
if score >= 90:  
    print('优秀')  
elif score >= 80:  
    print('良好')
```

不再
执行

```
elif score >= 60:  
    print('及格')  
else:  
    print('不及格')
```

良好

只有 `if` (没有 `elif` 和 `else`):

```
python  
  
if score >= 60:  
    print('及格')
```

只有 `if` 和 `else`:

```
python  
  
if score >= 60:  
    print('及格')  
else:  
    print('不及格')
```

分支语句

- 分支语句用于根据判断条件选择程序执行路径

```
password = input('请输入密码: ')
```

```
if password == '123456':  
    print('登录成功')  
else:  
    print('密码错误')
```

= 赋值：把右边的值存进变量

== 判断：两边是不是相等

range()函数

- range() 用来产生一串**连续的整数**，最常见的用途是控制循环次数

想让一件事重复做 5 次，难道要这样写？

```
print('你好')
print('你好')
print('你好')
print('你好')
print('你好')
```



```
# 用 range 一行搞定
for i in range(5):
    print('你好')
```



range(5) 产生从 **0** 开始，到 **5 之前** 结束，一共 **5** 个数

```
print(list(range(5)))
# [0, 1, 2, 3, 4]
```

range()函数

■ 函数语法：

➤ range(stop)

➤ range(start, stop[, step])

- start: 计数从start开始，默认为0
- stop: 计数到stop结束，但不包括stop
- step: 步长，默认为1

```
print(list(range(5)))          # 一个参数：0 到 4  
# [0, 1, 2, 3, 4]
```

```
print(list(range(1, 6)))       # 两个参数：从 1 到 5  
# [1, 2, 3, 4, 5]
```

```
print(list(range(1, 10, 2)))    # 三个参数：从 1 到 9，每次加 2  
# [1, 3, 5, 7, 9]
```

```
print(list(range(10, 0, -1)))   # 步长为负：倒着数  
# [10, 9, 8, 7, 6, 5, 4, 3, 2, 1]
```

循环语句

- 根据判断条件或计数条件确定一段程序的运行次数
- 循环语句有两种类型：

➤ 遍历循环：for语句

for *i* in range (<计数值>):

<表达式1>

```
for i in range(10):  
    print(i)
```

➤ 条件/无限循环：while语句

while <条件>:

<表达式1>

```
i = 0  
while i < 10:  
    print(i)  
    i = i + 1
```

循环语句

■ 何时选择 for 还是 while ?

	for	while
适合	知道要循环多少次	不知道要循环多少次
例子	打印 10 遍、遍历一个字符串	一直问用户密码，直到输对为止
计数	自动完成	要自己写 <code>i = i + 1</code>

- while 死循环

```
i = 0
```

```
while i < 10:
```

```
    print(i)
```

```
    # 忘了写 i = i + 1
```

while 循环里，一定要有能让条件变为不成立的语句，否则就是死循环！

i 永远是 0，条件永远成立，程序停不下来

实例解析-温度转换

- 背景：温度有摄氏度(Celsius)和华氏度(Fahrenheit)两个度量体系，不同国家采用不同的温度表示方法
- 问题的IPO描述：
 - 输入(input)：带华氏度或摄氏度标志的温度值
 - 处理(process)：根据温度标志选择适当的温度转换算法
 - 输出(output)：带摄氏或华氏标志的温度值

华氏度和摄氏度的转换方法： $C = (F - 32) / 1.8$
 $F = C * 1.8 + 32$

C 表示摄氏温度，如 28C 表示 28 摄氏度
F 表示华氏温度，如 82F 表示 82 华氏度



实例解析-温度转换

■ 编写程序

```
TempStr = input('请输入带有符号的温度值: ')    # 例如输入 82F 或 28C

if TempStr[-1] in ['F', 'f']:                    # 取最后一个字符, 判断是不是 F
    C = (float(TempStr[0:-1]) - 32) / 1.8        # 去掉末尾字母, 转成小数后按公式计算
    print('转换后的温度为 {:.2f}C'.format(C))    # 保留两位小数输出

elif TempStr[-1] in ['C', 'c']:                  # 否则判断是不是 C
    F = 1.8 * float(TempStr[0:-1]) + 32          # 摄氏转华氏
    print('转换后的温度为 {:.2f}F'.format(F))

else:                                             # 两种都不是
    print('输入格式错误')
```

实例解析-BMI 体重指数计算

背景 BMI（身体质量指数）是常用的体重健康评估指标，计算公式为：

$$\text{BMI} = \frac{\text{体重(kg)}}{\text{身高(m)}^2}$$

分级标准：

BMI 范围	结论
小于 18.5	体重偏轻
18.5 ~ 24	体重正常
24 ~ 28	体重偏重
28 及以上	肥胖

要求

编写程序，连续为 3 位用户测量 BMI：

1. 每位测量前提示"第 N 位"
2. 依次输入身高（米）和体重（千克）
3. 计算 BMI 并输出，保留两位小数
4. 给出对应的分级结论
5. 三位全部测量完毕后，输出"三位全部测量完毕"

实例解析-BMI 体重指数计算

■ 编写程序

```
for i in range(3):                                # 固定测 3 个人
    print('--- 第', i + 1, '位 ---')              # i 从 0 开始, 所以显示 i+1
    height = float(input('请输入身高 (米): '))
    weight = float(input('请输入体重 (千克): '))
    bmi = weight / (height * height)
    print('您的BMI值为{:.2f}'.format(bmi))

    if bmi < 18.5:
        print('体重偏轻')
    elif bmi < 24:
        print('体重正常')
    elif bmi < 28:
        print('体重偏重')
    else:
        print('肥胖')

print('三位全部测量完毕')
```

顶格写, 循环结束后才执行